

Arquitectura y Diseño del Sistema Proxy con Rate Limiting

Índice

1. [Visión General](#)
 2. [Arquitectura del Sistema](#)
 3. [Componentes Principales](#)
 4. [Patrones de Diseño Implementados](#)
 5. [Flujo de una Request](#)
 6. [Rate Limiting - Diseño Detallado](#)
 7. [Escalabilidad Horizontal](#)
 8. [Consideraciones de Performance](#)
 9. [Diagramas](#)
-

Visión General

Este sistema es un **API Gateway Proxy** reactivo construido con Spring Boot WebFlux que actúa como intermediario entre clientes y APIs upstream (principalmente `api.mercadolibre.com`). Su función principal es:

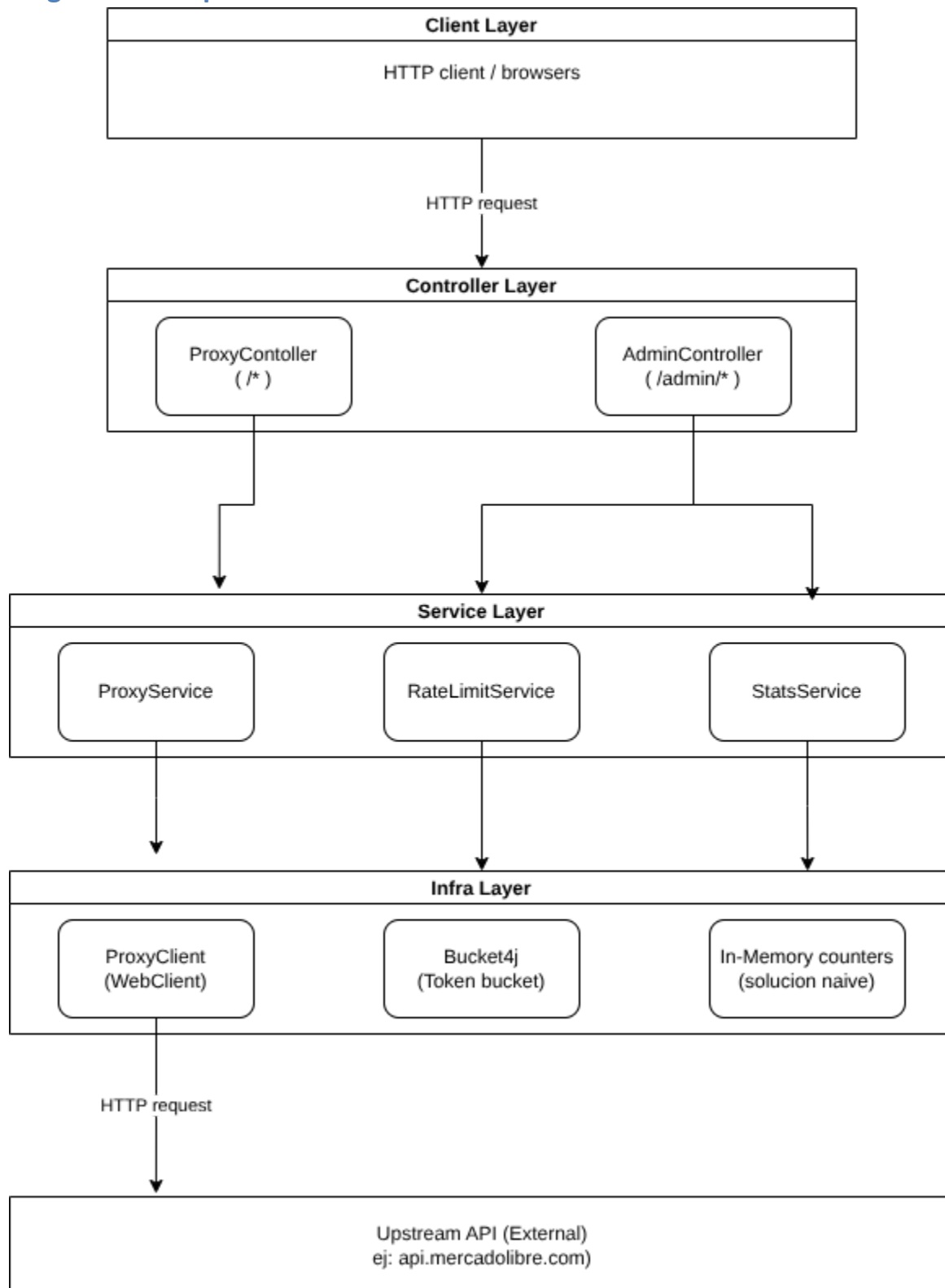
- **Proxy transparente:** Redirige todas las requests al servidor upstream configurado
- **Rate Limiting configurable:** Aplica límites de tasa por IP, path, combinaciones IP+path o globalmente
- **Observabilidad:** Recolecta estadísticas de tráfico y errores
- **Administración dinámica:** Permite agregar/eliminar reglas de rate limiting en tiempo de ejecución

Stack Tecnológico

- **Java 21**
 - **Spring Boot 3.2 (WebFlux)** - Stack reactivo no bloqueante
 - **Bucket4j** - Algoritmo Token Bucket para rate limiting
 - **Reactor Netty** - Cliente/servidor HTTP asíncrono
 - **Maven** - Gestión de dependencias
-

Arquitectura del Sistema

Diagrama de Capas



Arquitectura Reactiva (WebFlux)

El sistema utiliza programación reactiva con **Project Reactor** (Mono y Flux):

- **Non-blocking I/O:** Cada request no bloquea un thread mientras espera la respuesta del upstream
 - **Backpressure:** Control automático de flujo para evitar sobrecarga
 - **High Throughput:** Capaz de manejar miles de requests concurrentes con pocos threads
-

Componentes Principales

1. Controller Layer

ProxyController

```
@RestController
public class ProxyController {
    @RequestMapping("/**")
    public Mono<ResponseEntity<byte[]>> proxy(...)
}
```

Responsabilidades: - Captura **todas** las requests (/**) - Delega inmediatamente a ProxyService - Mantiene el controller “thin” (sin lógica de negocio)

AdminController

```
@RestController
@RequestMapping("/admin")
public class AdminController {
    @GetMapping("/stats")
    @GetMapping("/rules")
    @PostMapping("/rules")
    @DeleteMapping("/rules/{id}")
}
```

Responsabilidades: - Gestión de reglas de rate limiting - Consulta de estadísticas - Protegido con HTTP Basic Authentication

Patrón: *REST API Controller*

2. Service Layer

ProxyService

Responsabilidades principales:

1. **Filtrado de rutas administrativas:** Excluye /admin/* y /actuador/* del proxy
2. **Rate Limiting:** Consulta al RateLimitService si la request está permitida
3. **Registro de estadísticas:** Invoca a StatsService
4. **Construcción del URI upstream:** Preserva path y query parameters
5. **Filtrado de headers:** Elimina headers “hop-by-hop” (Connection, Keep-Alive, etc.)
6. **Delegación al cliente upstream:** Invoca ProxyClient.execute()
7. **Manejo de errores:** Convierte excepciones en responses 502 Bad Gateway
8. **Content-Type por defecto:** Aplica un content-type configurable si el upstream no lo provee

Patrones: - *Service Layer Pattern*: Encapsula lógica de negocio - *Coordinador*: Orquesta múltiples servicios

```
public Mono<ResponseEntity<byte[]>> forward(ServerWebExchange exchange,
Mono<byte[]> body) {
    // 1. Filtrar admin routes
    // 2. Rate limit check
    // 3. Stats recording
    // 4. Build URI + filter headers
    // 5. Execute upstream call
    // 6. Map response
    // 7. Handle errors
}
```

RateLimitService

Estructura de datos:

```
// Master map: todas las reglas por ID
ConcurrentMap<String, RateLimitRule> rulesById;

// Índices para búsqueda O(1):
ConcurrentMap<String, RateLimitRule> ipIndex;           // key: "1.2.3.4"
ConcurrentMap<String, RateLimitRule> pathExactIndex;    // key: "/sites"
ConcurrentMap<String, RateLimitRule> pathPrefixIndex;   // key: "/sites/*"
ConcurrentMap<String, RateLimitRule> ipPathIndex;       // key:
"1.2.3.4/sites"

// Regla global única (singleton)
volatile RateLimitRule globalRule;

// Buckets de Bucket4j por ruleId
ConcurrentMap<String, Bucket> buckets;
```

Responsabilidades:

1. **Gestión de reglas:** addRule(), removeRuleById(), getRules()

2. **Indexación eficiente:** Delega a estrategias polimórficas
3. **Evaluación de rate limit:** tryConsume(ip, path)
 - Recolecta candidatos desde los índices
 - Elimina duplicados
 - Evalúa reglas (de más específica a más general)
 - Consume tokens del bucket correspondiente
4. **Gestión de buckets:** Crea buckets con configuración Token Bucket de Bucket4j

Algoritmo de evaluación:

tryConsume(ip, path):

1. Buscar reglas candidatas:
 - IP match exacto
 - PATH match exacto
 - PATH match con wildcard (prefix)
 - IP_PATH match exacto
 - IP_PATH match con wildcard
 - GLOBAL rule
2. Eliminar duplicados (misma regla)
3. Para cada candidato (orden de especificidad):
 - Verificar si la regla matchea (ip, path)
 - Obtener o crear bucket para esa regla
 - Intentar consumir 1 token
 - Si falla → DENIED (429 Too Many Requests)
 - Si todas las reglas permiten → ALLOWED
4. Si no hay reglas aplicables → ALLOWED

Patrones: - *Strategy Pattern*: Delegación a RateLimitStrategy - *Index Approach*: Múltiples índices para búsqueda $O(1)$ - *Token Bucket Algorithm*: Rate limiting con ráfagas permitidas

StatsService

Métricas recolectadas:

// Contadores globales

totalRequests, allowedRequests, deniedRequests

upstreamClientErrors (4xx), upstreamServerErrors (5xx)

// Contadores por dimensión

byPath, byIp

upstream4xxByPath, upstream4xxByIp

upstream5xxByPath, upstream5xxByIp

Uso de LongAdder: Thread-safe y optimizado para alta concurrencia (mejor que AtomicLong bajo contención)

3. Client Layer

ProxyClient (Interface)

```
public interface ProxyClient {  
    Mono<ProxyResponse> execute(HttpMethod method, String uri,  
                                HttpHeaders headers, Mono<byte[]> body);  
}
```

Permite inyectar implementaciones diferentes: - DefaultProxyClient: Cliente HTTP real (producción) - MockProxyClient: Cliente mock para testing

DefaultProxyClient

```
public class DefaultProxyClient implements ProxyClient {  
    private final WebClient webClient;  
  
    public Mono<ProxyResponse> execute(...) {  
        return webClient.method(method)  
            .uri(uri)  
            .headers(...)  
            .body(...)  
            .exchangeToMono(...)  
            .onErrorResume(...);  
    }  
}
```

Características: - Usa WebClient de Spring WebFlux (cliente HTTP reactivo) - No sigue redirects (followRedirect(false)) - Retorna ProxyResponse con status, headers y body - Maneja errores de conexión retornando 502

Patrón: *Adapter Pattern* (adapta WebClient a ProxyClient)

4. Rate Limiting Module

RateLimitRule

```
public final class RateLimitRule {  
    private final String id; // UUID único  
    private final RateLimitType type; // IP, PATH, IP_PATH, GLOBAL  
    private final String ip; // null si no aplica  
    private final String path; // null si no aplica  
    private final long rpm; // Requests Per Minute  
}
```

Inmutabilidad: Clase final con campos final → Thread-safe por diseño

RateLimitType (Enum + Strategy)

```
public enum RateLimitType {  
    IP(new IpStrategy()),  
    PATH(new PathStrategy()),  
    IP_PATH(new IpPathStrategy()),  
    GLOBAL(new GlobalStrategy());  
  
    private final RateLimitStrategy strategy;  
}
```

Patrón: *Strategy Pattern + Enum with Behavior*

Cada tipo delega su comportamiento a una estrategia específica:

Estrategia	Valida	Matchea	Compute Key	Index
IpStrategy	Requiere ip	ip == requestIp	ip	ipIndex
PathStrategy	Requiere path	Exact o wildcard match	path	pathExactIndex o pathPrefixIndex
IpPathStrategy	Requiere ip y path	Ambos deben matchear	ip path	ipPathIndex
GlobalStrategy	No requiere nada	Siempre matchea	"GLOBAL"	N/A

RateLimitStrategy (Interface)

```
public interface RateLimitStrategy {  
    void validate(RateLimitRuleRequest req);  
    boolean isGlobal();  
    RateLimitRule index(...); // Retorna regla previa si existe  
    void removeFromIndex(...);  
    boolean matches(RateLimitRule rule, String ip, String path);  
    String computeKey(RateLimitRule rule, String ip, String path);  
}
```

Ventajas del diseño: - **Open/Closed Principle:** Podes agregar nuevos tipos sin modificar RateLimitService - **Single Responsibility:** Cada estrategia maneja su propia lógica - **Testable:** Cada estrategia se puede testear independientemente

Wildcard Path Matching

Las reglas de path soportan wildcards:

/sites/* → matchea /sites/MLA, /sites/MLB, etc.
/items → solo matchea /items exacto

Implementación:

```
private boolean pathMatches(String pattern, String actualPath) {  
    if (pattern == null) return false;  
    if (pattern.endsWith("/*")) {  
        String prefix = pattern.substring(0, pattern.length() - 2);  
        return actualPath != null && actualPath.startsWith(prefix);  
    }  
    return pattern.equals(actualPath);  
}
```

RateLimitRuleManager

Responsabilidades: - Validar requests de creación de reglas (delegando a RateLimitType)
- Crear instancias de RateLimitRule con ID único

Patrón: *Factory Pattern + Validator Pattern*

```
public RateLimitRule createRule(RateLimitRuleRequest req) {  
    req.getType().validate(req); // Delega validación a la estrategia  
    return new RateLimitRule(  
        req.getType(),  
        req.getIp(),  
        req.getPath(),  
        req.getRpm()  
    );  
}
```

5. Configuration Layer

ProxyConfig

@Configuration

```
public class ProxyConfig {  
    @Bean  
    public WebClient upstreamWebClient() {  
        return WebClient.builder()  
            .baseUrl(targetBaseUrl)  
            .clientConnector(...)   
            .build();  
    }  
}
```



```

@Bean
@ConditionalOnMissingBean(proxyClient.class)
public ProxyClient proxyClient(WebClient upstreamWebClient) {
    return new DefaultProxyClient(upstreamWebClient);
}
}

```

Patrón: *Dependency Injection + Configuration as Code*

SecurityConfig

```

@Configuration
@EnableWebFluxSecurity
public class SecurityConfig {
    @Bean
    public SecurityWebFilterChain securityWebFilterChain(ServerHttpSecurity
http) {
        return http
            .authorizeExchange(exchanges -> exchanges
                .pathMatchers("/admin/**").authenticated()
                .anyExchange().permitAll()
            )
            .httpBasic(Customizer.withDefaults())
            .csrf(ServerHttpSecurity.CsrfSpec::disable)
            .build();
    }
}

```

Seguridad: - /admin/** protegido con HTTP Basic Auth - Resto del tráfico sin autenticación (proxy público)

MockClientConfig

```

@Configuration
@Profile("mock-client")
public class MockClientConfig {
    @Bean
    public ProxyClient proxyClient() {
        return new MockProxyClient();
    }
}

```

Activar con: `--spring.profiles.active=mock-client`

Patrones de Diseño Implementados

1. Strategy Pattern

Dónde: RateLimitStrategy + implementaciones (IpStrategy, PathStrategy, etc.)

Por qué: - Cada tipo de rate limit tiene lógica diferente - Permite agregar nuevos tipos sin modificar código existente - Facilita testing unitario de cada estrategia

2. Dependency Injection (DI)

Dónde: Todo el proyecto (Spring IoC Container)

Beneficios: - Bajo acoplamiento - Fácil testing (puedo inyectar mocks) - Configuración centralizada

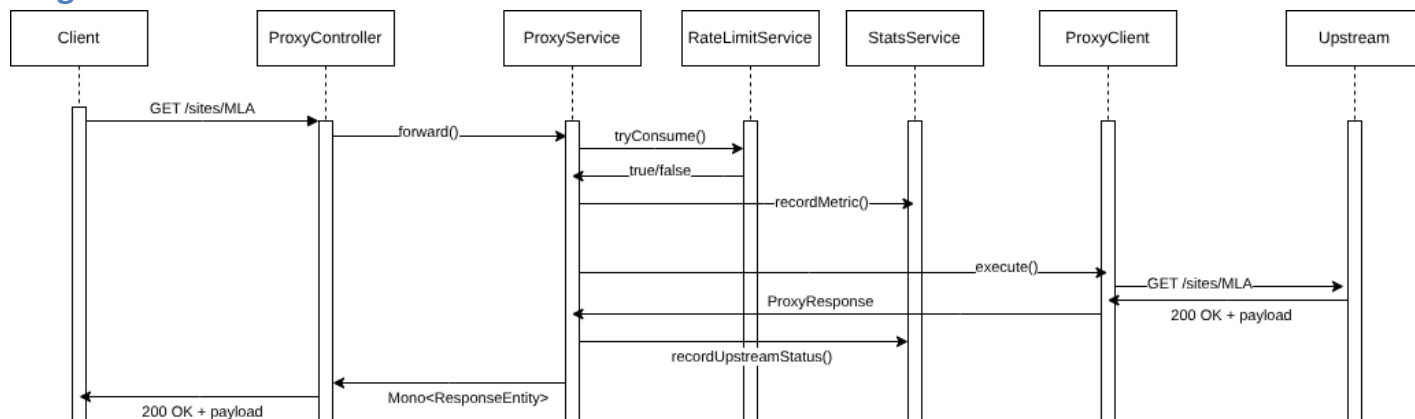
3. Adapter Pattern

Dónde: DefaultProxyClient adapta WebClient a ProxyClient

Por qué: - Abstrae la implementación concreta del cliente HTTP - Permite cambiar la biblioteca subyacente sin afectar el resto del código

Flujo de una Request

Diagrama de Secuencia



Paso a paso

1. Cliente envía request → GET http://localhost:8080/sites/MLA

2. **ProxyController captura** → @RequestMapping("/**")

3. **ProxyService.forward()**

- Extrae IP del cliente
- Verifica si es ruta administrativa → NO
- Llama a rateLimitService.tryConsume(ip, path)

4. **RateLimitService.tryConsume()**

ip: "192.168.1.100"
path: "/sites/MLA"

Búsqueda de reglas candidatas:

- ipIndex["192.168.1.100"] → null
- pathExactIndex["/sites/MLA"] → null
- pathPrefixIndex["/sites/*"] → Rule{id="...", type=PATH, path="/sites/*", rpm=100}
- ipPathIndex["192.168.1.100|/sites/MLA"] → null
- globalRule → null

Candidatos: [Rule{PATH, /sites/*, 100}]

Evaluación:

- rule.matches("192.168.1.100", "/sites/MLA") → TRUE
- Obtener bucket para rule.id
- bucket.tryConsume(1) → TRUE (hay tokens disponibles)

Resultado: ALLOWED

5. **StatsService.record()**

```
totalRequests++
allowedRequests++
byPath["/sites/MLA"]++
byIp["192.168.1.100"]++
```

6. **ProxyService construye URI**

```
baseUrl: "http://api.mercadolibre.com"
path: "/sites/MLA"
→ uri: "http://api.mercadolibre.com/sites/MLA"
```

7. **HeaderUtils.filterHopByHop()**

Elimina: Connection, Keep-Alive, Proxy-Authenticate, Proxy-Authorization, TE, Trailers, Transfer-Encoding, Upgrade

8. **ProxyClient.execute()**

```
WebClient.get()
    .uri("/sites/MLA")
```

```
.headers(filtered)
.retrieve()
```

9. Upstream responde

```
Status: 200
Headers: {Content-Type: application/json, ...}
Body: {"id":"MLA","name":"Argentina",...}
```

10. ProxyResponseMapper.map()

Convierte ProxyResponse → ResponseEntity<byte[]>
Aplica defaultContentType si no hay Content-Type

11. StatsService.recordUpstreamStatus()

(status 200 → no incrementa errores)

12. ProxyController retorna → Mono<ResponseEntity<byte[]>>

13. Cliente recibe respuesta → 200 OK con body JSON

Flujo con Rate Limit Excedido

Cliente → ProxyController → ProxyService → RateLimitService

```

                                     |
                                     | tryConsume() → FALSE
                                     |
StatsService <-----| record(denied=true)
```

Cliente ← 429 TOO_MANY_REQUESTS

Response:

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Content-Length: 51
```

```
{"error":"Rate limit exceeded. Try again later."}
```

Flujo con Error Upstream

ProxyService → ProxyClient → Upstream

```

                                     |
                                     | [Connection timeout / 500 Error]
                                     |
                                     | onErrorResume()
                                     |
                                     |> ProxyResponse(status=502, body="upstream error: ...")
```

ProxyService → StatsService.recordUpstreamStatus(502)
|
Cliente ← 502 BAD_GATEWAY

Rate Limiting - Diseño Detallado

Token Bucket Algorithm (Bucket4j)

Concepto: - Cada regla tiene un “bucket” (cubeta) con capacidad = rpm - Se rellenan tokens a razón de rpm tokens/minuto - Cada request consume 1 token - Si no hay tokens → request denegada

Configuración:

```
Bandwidth limit = Bandwidth.classic(  
    rpm, // capacidad  
    Refill.greedy(rpm, Duration.ofMinutes(1)) // recarga  
);  
Bucket bucket = Bucket.builder().addLimit(limit).build();
```

Ventajas: - Permite ráfagas (burst): Si no hay tráfico, se acumulan tokens hasta rpm - Distribución uniforme: Los tokens se recargan continuamente

Jerarquía de Reglas y Precedencia

Orden de especificidad (de más específico a menos):

1. **IP_PATH con path exacto:** "192.168.1.1" | "/sites"
2. **IP_PATH con wildcard:** "192.168.1.1" | "/sites/*"
3. **PATH exacto:** "/sites"
4. **PATH wildcard:** "/sites/*"
5. **IP:** "192.168.1.1"
6. **GLOBAL:** Aplica a todas las requests

Regla de evaluación: - **TODAS** las reglas aplicables deben permitir la request - Si **UNA SOLA** deniega → 429 Too Many Requests

Ejemplo:

Reglas configuradas:

1. PATH "/sites/*" → 1000 RPM
2. IP_PATH "1.2.3.4" | "/sites/*" → 10 RPM
3. GLOBAL → 5000 RPM

Request: GET /sites/MLA desde IP 1.2.3.4

Evaluación:

- Regla 1 matchea → bucket tiene tokens → OK
- Regla 2 matchea → bucket tiene tokens → OK
- Regla 3 matchea → bucket tiene tokens → OK

Resultado: ALLOWED (200)

Pero si la IP 1.2.3.4 ya hizo 10 requests:

- Regla 2 matchea → bucket SIN tokens → DENIED

Resultado: 429 Too Many Requests

Implicación: Las reglas más específicas pueden **restringir** las más generales

Estrategia de Índices

Problema: Con miles de reglas, iterar todas en cada request sería $O(n)$ → lento

Solución: Múltiples índices para lookup $O(1)$

// Estructura

```
ConcurrentMap<String, RateLimitRule> ipIndex;  
ConcurrentMap<String, RateLimitRule> pathExactIndex;  
ConcurrentMap<String, RateLimitRule> pathPrefixIndex;  
ConcurrentMap<String, RateLimitRule> ipPathIndex;
```

Ejemplo:

Reglas:

- Rule1: IP "1.2.3.4" 100rpm
- Rule2: PATH "/sites" 500rpm
- Rule3: PATH "/items/*" 1000rpm
- Rule4: IP_PATH "5.6.7.8" | "/sites" 50rpm

Índices:

```
ipIndex = {  
  "1.2.3.4" → Rule1  
}
```

```
pathExactIndex = {  
  "/sites" → Rule2  
}
```

```
pathPrefixIndex = {  
  "/items/*" → Rule3  
}
```

```
ipPathIndex = {  
    "5.6.7.8|/sites" → Rule4  
}
```

Búsqueda para request GET /sites desde 1.2.3.4:

1. ipIndex.get("1.2.3.4") → Rule1 ✓
2. pathExactIndex.get("/sites") → Rule2 ✓
3. pathPrefixIndex → (iterar) ninguna matchea
4. ipPathIndex.get("1.2.3.4|/sites") → null
5. globalRule → null

Candidatos: [Rule1, Rule2]

Complejidad: O(1) para la mayoría + O(k) para prefixes (k = número de reglas con wildcard)

Reemplazo de Reglas

Comportamiento: Solo puede existir UNA regla por combinación de criterios

```
// Agregar regla PATH "/sites" 100rpm  
addRule(new RateLimitRule(PATH, null, "/sites", 100));  
  
// Agregar otra regla PATH "/sites" 200rpm  
addRule(new RateLimitRule(PATH, null, "/sites", 200));  
  
// Resultado: La primera regla es REEMPLAZADA por la segunda  
// Solo existe una regla en pathExactIndex["/sites"]
```

Implementación:

```
RateLimitRule previous = rule.getType().index(rule, ...);  
if (previous != null) {  
    rulesById.remove(previous.getId());  
    removeBucketForRule(previous.getId());  
    LOGGER.info("Replaced rule {} with {}", previous.getId(), rule.getId());  
}
```

Ventaja: Previene configuraciones inconsistentes

Regla GLOBAL (Singleton)

Especial: Solo puede haber UNA regla GLOBAL a la vez

```
private volatile RateLimitRule globalRule = null;
```

```

public void addRule(RateLimitRule rule) {
    if (rule.getType().isGlobal()) {
        RateLimitRule existing = this.globalRule;
        if (existing != null) {
            rulesById.remove(existing.getId());
            removeBucketForRule(existing.getId());
        }
        this.globalRule = rule;
        // ...
    }
}

```

Uso típico:

```

# Limitar TODO el tráfico a 10000 RPM
curl -X POST http://localhost:8080/admin/rules \
  -u admin:password123 \
  -H "Content-Type: application/json" \
  -d '{
    "type": "GLOBAL",
    "rpm": 10000
  }'

```

Escalabilidad Horizontal

Limitaciones Actuales (In-Memory)

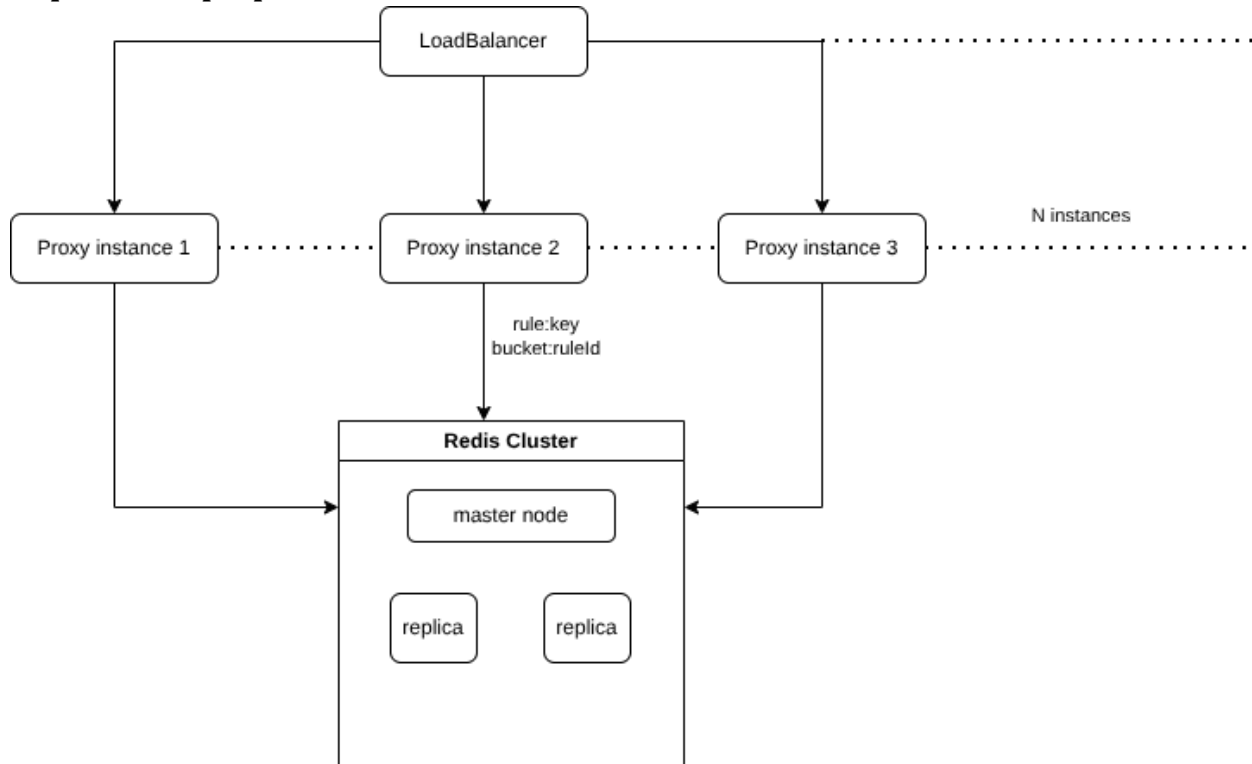
Estado local no compartido:

Problemas:

1. **Reglas inconsistentes:** Agregar una regla en Instance 1 no afecta Instance 2
2. **Rate limiting independiente:** Si la regla es 100 RPM, CADA instancia permite 100 RPM
→ Total: 300 RPM
3. **Estadísticas fragmentadas:** Las métricas están distribuidas

Solución: Redis + Bucket4j Distribuido

Arquitectura propuesta:



Script Lua en Redis (ejecutado por Bucket4j): - Bucket4j genera scripts Lua que se ejecutan atómicamente en Redis - Garantiza consistencia sin race conditions

Escalado con 50,000 RPS (Requests Per Second)

Escenario: - **Load objetivo:** 50,000 requests/segundo (RPS) - **Instancias requeridas:** 10-15 instancias (según hardware) - **Redis:** Cluster con nodos master (sharding) + réplicas

Análisis de Capacidad:

1. Capacidad por Instancia (Spring WebFlux)

Hardware de referencia: 8 CPU cores, 16GB RAM

Spring WebFlux (benchmarks de la comunidad):

- Con proxy + rate limiting + Redis: ~3,000-5,000 RPS/instancia (degradación por I/O)

Factores que impactan:

- Round-trip a Redis: 1-2ms en LAN
- Upstream API latency: 50-500ms (bloquea recursos aunque sea non-blocking)

Cálculo conservador:

- Asumiendo: ~3,000-4,000 RPS sostenible por instancia

Instancias necesarias para 50,000 RPS:

50,000 RPS / 3,500 RPS por instancia $\approx$ 14-15 instancias
+ Factor de seguridad 20% $\rightarrow$ ~18 instancias recomendadas

2. Capacidad de Redis (falta analisis)

Redis single-threaded performance (documentación oficial Redis Labs):

- GET/SET simples: 100,000+ ops/seg
- Scripts Lua (Bucket4j): 50,000-80,000 ops/seg (más pesados)

Operaciones por request en nuestro sistema:

- 1-2 ops para rate limit (EVALSHA del script Lua de Bucket4j)
- Total: ~3-6 ops Redis por request

Cálculo de carga Redis:

50,000 RPS $\times$ 4 ops promedio = 200,000 ops/seg

Capacidad de 1 nodo Redis: ~60,000 ops/seg (scripts Lua)

Nodos master necesarios: 200,000 / 60,000 $\approx$ 4 nodos

Arquitectura recomendada:

- 4-6 nodos master (sharding por hash de key)
- 1-2 réplicas por master (lectura + HA)
- Total: 12-18 nodos Redis

Consideraciones de Performance

1. WebFlux Non-Blocking

Ventaja: Un thread pool pequeño puede manejar miles de requests concurrentes

Clave: Mientras espera respuesta del upstream, el thread se libera para procesar otras requests

2. Estructuras de Datos Thread-Safe

ConcurrentHashMap // *O(1) Lookup, Lock striping*
LongAdder // *Mejor que AtomicLong bajo alta contención*
volatile // *Visibility garantizada sin lock (para globalRule)*

3. Circuit Breaker (futuro)

Problema: Si el upstream está caído, seguimos intentando conectar → desperdicio de recursos

Diagramas

Diagrama de Estados del Rate Limiter

Token Bucket State

Estado Inicial:

Bucket: 100/100 | (capacidad completa: 100 tokens)

Request 1:

Bucket: 99/100 | → ALLOWED (consume 1 token)

Request 2:

Bucket: 98/100 | → ALLOWED

... (97 requests más)

Request 100:

Bucket: 0/100 | → ALLOWED (último token)

Request 101:

Bucket: 0/100 | → DENIED (sin tokens)

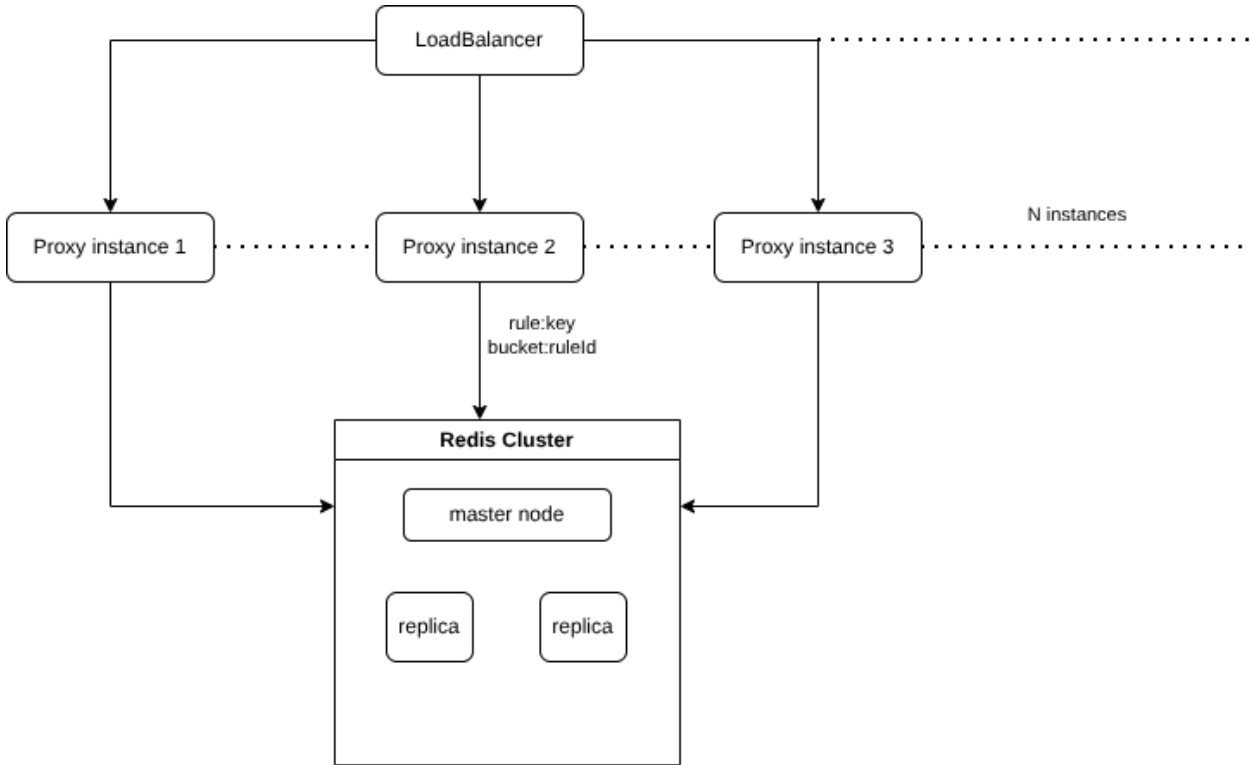
└─> Return: 429 Too Many Requests

Después de 36 segundos (60% de 1 minuto):

Bucket: 60/100 | (refill proporcional)

Request 102:

Diagrama de Deployment (Escalado Horizontal)



Resumen de Decisiones Técnicas

Aspecto	Decisión	Razón
Framework	Spring Boot WebFlux	Non-blocking I/O, alto throughput
Rate Limiting	Bucket4j Token Bucket	Algoritmo estándar, permite bursts
Concurrencia	ConcurrentHashMap, LongAdder	Thread-safe, alto performance
Client HTTP	WebClient (Reactor Netty)	Reactivo, compatible con WebFlux
Estrategias	Strategy Pattern	Extensible, testeable
Índices	Múltiples mapas por tipo	Lookup O(1) vs O(n)
Reglas únicas	Una regla por clave	Previene inconsistencias

Headers	Filtrar hop-by-hop	Evita problemas de proxy
Errores upstream	502 Bad Gateway	Estándar HTTP para proxy errors
Autenticación admin	HTTP Basic	Simple, suficiente para admin endpoints (version naive)
Testing	Mock client + profiles	Permite tests sin dependencias externas
Escalabilidad	Redis para estado compartido	Distributed rate limiting

Próximos Pasos y Mejoras

1. **Circuit Breaker** (Resilience4j) para manejo de fallos upstream
 2. **Redis distribuido** para escalado horizontal real
 3. **Métricas más completas (Prometheus/new Relic/Grafana/etc)**
 4. **Health checks** (/actuator/health)
 5. **Rate limit por API key** (además de IP)
 6. **Caché de responses** para reducir carga en upstream
 7. **Autenticación OAuth2/API key** para endpoints protegidos
-

Conclusión

Este sistema implementa un **API proxy** con las siguientes características destacadas:

- ✓ **Arquitectura reactiva** para alto throughput
- ✓ **Rate limiting flexible** con 4 estrategias
- ✓ **Thread-safe** con estructuras concurrentes
- ✓ **Testeable** con mocks y profiles
- ✓ **Preparado para escalar** (con Redis)
- ✓ **Observabilidad** con stats y logs